BOOTCAMP · DATA SCIENCE & FULL STACK · MÓDULO SQL

Proyecto de Creación de una Base de Datos Relacional

Explicación paso a paso: de unos CSV sin normalizar a una base de datos PostgreSQL alojada en la nube y lista para consultar.

Equipo E	Noa, Patricio, Nicolás y Paula (DS) · Mariela Velasquez Canache y Carlos Rodríguez Ruiz (FS)
Tipo de trabajo	Proyecto grupal
Datos de partida	5 ficheros CSV: `clase_1` … `clase_4` y `claustro`
Herramientas	draw.io · MySQL Workbench · PostgreSQL (Supabase)
Resultado	10 tablas · 500 filas · consultas de demostración

Índice

0 Antes de empezar: qué pide el proyecto y vocabulario básico

1 Los datos de partida

2 Paso 1 · Normalización y modelo Entidad-Relación

3 Paso 2 · Modelo lógico (relacional)

4 Paso 3 · Verificación de la normalización

5 Paso 4 · Creación de la base de datos (MySQL y PostgreSQL en Supabase)

6 Consultas de demostración

7 Escalabilidad: cómo crece la base de datos

8 Decisiones, limitaciones y mejoras futuras

A Anexo: ficheros entregados

> **Cómo leer este documento**
> Está pensado para que cualquier miembro del grupo pueda entender y defender el proyecto en la presentación, aunque no haya hecho la parte técnica. Cada paso explica **qué** se hace, **por qué** y **con qué resultado**, usando siempre nuestros datos reales.

0 · Antes de empezar

Qué nos pide el proyecto

Nos dan los datos de los alumnos de varias promociones del bootcamp (y de sus profesores) en ficheros CSV **sin normalizar**: todo mezclado en una tabla grande, con muchos datos repetidos. Tenemos que convertirlos en una **base de datos relacional** bien diseñada, que no repita información, que sea capaz de crecer (más campus, verticales, promociones, modalidades…) y que esté alojada en un servidor para poder consultarla desde otras aplicaciones.

Tarea del enunciado	Qué entregamos
1. Modelo Entidad-Relación (E/R)	Diagrama E/R hecho en draw.io
2. Modelo lógico	Esquema relacional: tablas, columnas, claves primarias y foráneas
3. Normalización	Análisis de la tabla original y comprobación de 1FN, 2FN y 3FN
4. Creación de la base de datos	Scripts SQL (MySQL y PostgreSQL) + base de datos en Supabase + consultas

> **Sobre el orden de los pasos**
>
> El enunciado pone la normalización en tercer lugar. Ese orden es el "clásico" de los libros, pensado para cuando *no* tienes datos y diseñas desde cero. Nosotros **sí** partimos de una tabla con datos, así que lo natural es **normalizar primero**: de la normalización salen solas las entidades del E/R. Por eso trabajamos así:
>
> 1. **Paso 1:** normalizamos la tabla original y de ahí sacamos el modelo E/R.
> 2. **Paso 2:** pasamos el E/R a tablas (modelo relacional).
> 3. **Paso 3:** volvemos a comprobar la normalización sobre las tablas finales (lo pide el enunciado y confirma que el diseño es correcto).
> 4. **Paso 4:** creamos la base de datos y cargamos los datos.

Vocabulario mínimo

Término	Qué significa (con un ejemplo nuestro)
Entidad	Una "cosa" de la que queremos guardar información. Ej.: *Persona*, *Promoción*, *Proyecto*. Se convierte en una tabla.
Atributo	Una característica de una entidad. Ej.: el *email* de un alumno. Se convierte en una columna.
Relación	Cómo se conectan dos entidades. Ej.: un alumno *realiza* proyectos.
Cardinalidad	Cuántos elementos participan en una relación. **1:N** = uno con muchos; **N:M** = muchos con muchos (un alumno hace muchos proyectos y un proyecto lo hacen muchos alumnos).
Clave primaria (PK)	Columna (o columnas) que identifica cada fila de forma única. Ej.: `persona.id`.
Clave foránea (FK)	Columna que apunta a la PK de otra tabla, y así "enlaza" tablas. Ej.: `alumnoProyecto.id_alumno` apunta a `alumnos.id_persona`.
Herencia (generalización)	Una entidad general con atributos comunes de la que cuelgan otras más concretas. Ej.: *Alumno* y *Profesor* son *Personas*: heredan su nombre y su email.
Atributo derivado	Un dato que se puede calcular a partir de otro, así que no se guarda. Ej.: el mes de la promoción sale de su fecha.
Redundancia	Guardar el mismo dato varias veces. Es lo que queremos eliminar.
Normalización	Proceso por pasos (1FN, 2FN, 3FN) para repartir los datos en tablas sin redundancia.

1 · Los datos de partida

Tenemos 5 ficheros CSV (separados por `;`) en la carpeta `data/`:

Fichero	Filas	Promoción	Campus	Proyectos que aparecen como columnas
`clase_1.csv`	15	Septiembre (18/09/2023)	Madrid	HLF, EDA, BBDD, ML, Deployment (Data Science)
`clase_2.csv`	10	Febrero (12/02/2024)	Madrid	HLF, EDA, BBDD, ML, Deployment (Data Science)
`clase_3.csv`	14	Septiembre (18/09/2023)	Madrid	WebDev, FrontEnd, Backend, React, FullSatck (Full Stack)
`clase_4.csv`	13	Febrero (12/02/2024)	Valencia	WebDev, FrontEnd, Backend, React, FullSatck (Full Stack)
`claustro.csv`	10	Profesores: nombre, rol (TA / LI), vertical, promoción, campus y modalidad		

Así es un CSV de alumnos (primeras filas de `clase_1.csv`)

Nombre	Email	Promoción	Fecha_comienzo	Campus	Proyecto_HLF	Proyecto_EDA	Proyecto_BBDD	Proyecto_ML	Proyecto_Deployment
Jafet Casals	Jafet_Casals@gmail.com	Septiembre	18/09/2023	Madrid	Apto	No Apto	Apto	Apto	Apto
Jorge Manzanares	Jorge_Manzanares@gmail.com	Septiembre	18/09/2023	Madrid	Apto	No Apto	Apto	Apto	Apto
Onofre Adadia	Onofre_Adadia@gmail.com	Septiembre	18/09/2023	Madrid	Apto	Apto	Apto	No Apto	Apto
… 12 filas más									

Así es el CSV del claustro (`claustro.csv`)

Nombre	Rol	Vertical	Promocion	Campus	Modalidad
Noa Yáñez	TA	DS	Septiembre	Madrid	Presencial
Saturnina Benitez	TA	DS	Septiembre	Madrid	Presencial
Mario Prats	LI	FS	Febrero	Valencia	Online
… 7 filas más · TA = Teacher Assistant, LI = Lead Instructor					

Cosas que nos fijamos al mirar los datos

- Una **promoción** queda definida por su **fecha de comienzo + campus + vertical**. Por ejemplo, "Septiembre 2023" existe en Madrid DS, Madrid FS y Valencia FS: son promociones distintas.
- La **vertical** (DS o FS) no viene escrita en los CSV de alumnos: se deduce de los proyectos (HLF, EDA… son de Data Science; WebDev, React… de Full Stack).
- El claustro menciona **dos promociones que no tienen alumnos** en los CSV: Valencia FS Septiembre y Madrid FS Febrero. Las creamos igualmente, porque tienen profesores.
- El nombre del proyecto `FullSatck` viene así (con la errata) en las cabeceras de los CSV y lo hemos mantenido tal cual.

2 · Paso 1: Normalización y modelo E/R

2.1 ¿Qué problemas tiene la tabla original?

Si metiéramos los CSV tal cual en una sola tabla, tendríamos esto (fíjate en lo que se repite):

Nombre	Email	Promoción	Fecha_comienzo	Campus	Proyecto_HLF	Proyecto_EDA	...
Jafet Casals	Jafet_Casals@...	Septiembre	18/09/2023	Madrid	Apto	No Apto	...
Jorge Manzanares	Jorge_Manzanares@...	Septiembre	18/09/2023	Madrid	Apto	No Apto	...
Onofre Adadia	Onofre_Adadia@...	Septiembre	18/09/2023	Madrid	Apto	Apto	...

Problema	Ejemplo con nuestros datos
Redundancia	"Septiembre · 18/09/2023 · Madrid" se repite en los 15 alumnos de `clase_1`.
Grupos repetidos	`Proyecto_HLF`, `Proyecto_EDA`... son el mismo dato (una nota) en 5 columnas. Si mañana hay un proyecto nuevo, hay que *cambiar la estructura* de la tabla.
Anomalía de actualización	Si la promoción cambia su fecha de inicio, hay que cambiarla en 15 filas. Si se olvida una, los datos se contradicen.
Anomalía de inserción	No podemos dar de alta la promoción "Valencia FS Septiembre" porque todavía no tiene alumnos.
Anomalía de borrado	Si borramos al último alumno de una promoción, perdemos también los datos de esa promoción.

La normalización resuelve todo esto en tres pasos. Cada forma normal (FN) exige cumplir la anterior.

2.2 Primera Forma Normal (1FN): un solo valor por celda, sin columnas repetidas

Regla: cada celda guarda un único valor y no hay grupos de columnas que signifiquen lo mismo.

Qué hacemos: las 5 columnas `Proyecto_X` se convierten en **filas**: una fila por cada alumno y proyecto.

Antes (1 fila por alumno, 5 columnas de notas)

Email	Proyecto_HLF	Proyecto_EDA	...
Jafet_Casals@...	Apto	No Apto	...

Después (1 fila por alumno y proyecto)

Email	Proyecto	Evaluación
Jafet_Casals@...	HLF	Apto
Jafet_Casals@...	EDA	No Apto
Jafet_Casals@...	BBDD	Apto

Ahora, añadir un proyecto nuevo es añadir filas, no columnas. También convertimos las fechas de texto (`18/09/2023`) al formato de fecha estándar (`2023-09-18`) para guardarlas como `DATE`.

2.3 Segunda Forma Normal (2FN): cada dato depende de la clave entera

Regla: si la clave de una tabla está formada por varias columnas, ningún otro dato puede depender solo de una parte de ella.

En la tabla de 1FN, la clave es **(alumno, proyecto)**: hacen falta las dos cosas para saber una evaluación. Pero hay datos que dependen solo de una parte:

- **Nombre y email** dependen solo del **alumno** → van a la tabla `alumnos`.
- **El nombre del proyecto** depende solo del **proyecto** → va a la tabla `proyecto`.
- **La evaluación** sí depende de los dos (de qué alumno y de qué proyecto) → se queda en la tabla intermedia `alumnoProyecto`.

alumnos		
id	**nombre**	**email**
1	Jafet Casals	Jafet_Casals@...
2	Jorge Manzanares	Jorge_Manzanares@...

proyecto	
id	**nombre**
1	HLF
2	EDA

alumnoProyecto		
id_alumno	**id_proyecto**	**evaluacion**
1	1	Apto
1	2	No Apto

Fíjate en que ya no escribimos "Jafet Casals" ni "HLF" muchas veces: usamos su **id**, que es un número corto.

2.4 Tercera Forma Normal (3FN): nada depende de algo que no sea la clave

Regla: un dato no puede depender de otro dato que no sea la clave (dependencia "transitiva").

En la tabla de alumnos todavía tendríamos promoción, fecha de comienzo y campus. Pero:

- La **fecha de comienzo**, el **campus** y la **vertical** no son datos del alumno: son datos de la **promoción**. Alumno → promoción → fecha. Esa cadena es la dependencia transitiva.
- Por eso sacamos todo eso a una tabla `promocion`, y relacionamos cada alumno con su promoción mediante la tabla `alumnosPromocion`.

- Igual con los profesores: su nombre, rol y modalidad van a su propia tabla; a qué promoción dan clase va en `profesorPromocion`.
- Y qué proyectos tiene cada promoción va en `promocionProyecto`.

promocion			
id	**fechaPromocion**	**campus**	**vertical**
1	2023-09-18	Madrid	DS
2	2024-02-12	Madrid	DS

alumnosPromocion	
id_alumno	**id_promocion**
1	1
2	1

> **Resultado de la normalización**
> "Madrid · 18/09/2023 · DS" se guarda **una sola vez** (en la promoción 1). Si cambia la fecha, se cambia en un único sitio. Podemos crear promociones sin alumnos, y borrar un alumno no borra su promoción.

2.5 De la normalización salen las entidades

Cada tabla "principal" que ha salido de normalizar es una **entidad** del modelo E/R. Las tablas intermedias son **relaciones**. Al normalizar nos salieron `alumnos` y `claustro`, pero las dos guardaban lo mismo: **nombre y email**. En lugar de repetirlo, lo subimos a una entidad general, **Persona**, y Alumno y Profesor **heredan** de ella.

Entidad	Atributos	De dónde sale
Persona	Id, Nombre, Email	Nombre y Email de `clase_X.csv`; nombre de `claustro.csv`
↳ **Alumno**	(hereda de Persona)	Cada fila de `clase_X.csv`
↳ **Profesor**	Rol, Modalidad (y hereda de Persona)	`claustro.csv`
Promoción	Id, Fecha promoción, Campus, Vertical, TipoModalidad, *MesPromoción (derivado)*	Columnas Promoción, Fecha_comienzo y Campus; la vertical se deduce de los proyectos
Proyecto	Id, Nombre	Cabeceras `Proyecto_X` de los CSV

Relación	Entre	Tipo	Se lee…
Participa	Alumno – Promoción	N:M	Un alumno participa en una o varias promociones; una promoción puede tener muchos alumnos (o ninguno todavía).
Enseña	Profesor – Promoción	N:M	Un profesor enseña en una o varias promociones; una promoción tiene varios profesores.
Realizan	Alumno – Proyecto	N:M	Un alumno realiza muchos proyectos; un proyecto lo realizan muchos alumnos. Tiene el atributo **Evaluación**.
Tienen	Promoción – Proyecto	N:M	Una promoción tiene varios proyectos; un proyecto se da en varias promociones.
Corrigen	Profesor – Proyecto	N:M	Un profesor corrige varios proyectos; todo proyecto lo corrige al menos un profesor.

> **La herencia es parcial y solapada**
> **Parcial:** puede haber personas que todavía no sean ni alumno ni profesor (por ejemplo, alguien que se ha dado de alta y aún no tiene promoción). **Solapada:** una misma persona puede ser alumno y profesor a la vez, como un alumno que hace de ayudante (TA) en otra promoción. Así su nombre y su email se guardan una sola vez.

> **¿Por qué todas las relaciones son N:M?**
> Con los datos actuales cada alumno está en una sola promoción, pero en la realidad un alumno puede hacer más de un bootcamp (por ejemplo, Data Science y después Full Stack), y un profesor puede corregir proyectos de otra promoción. Con N:M la base de datos ya está preparada para eso sin cambiar nada.

2.6 El diagrama Entidad-Relación

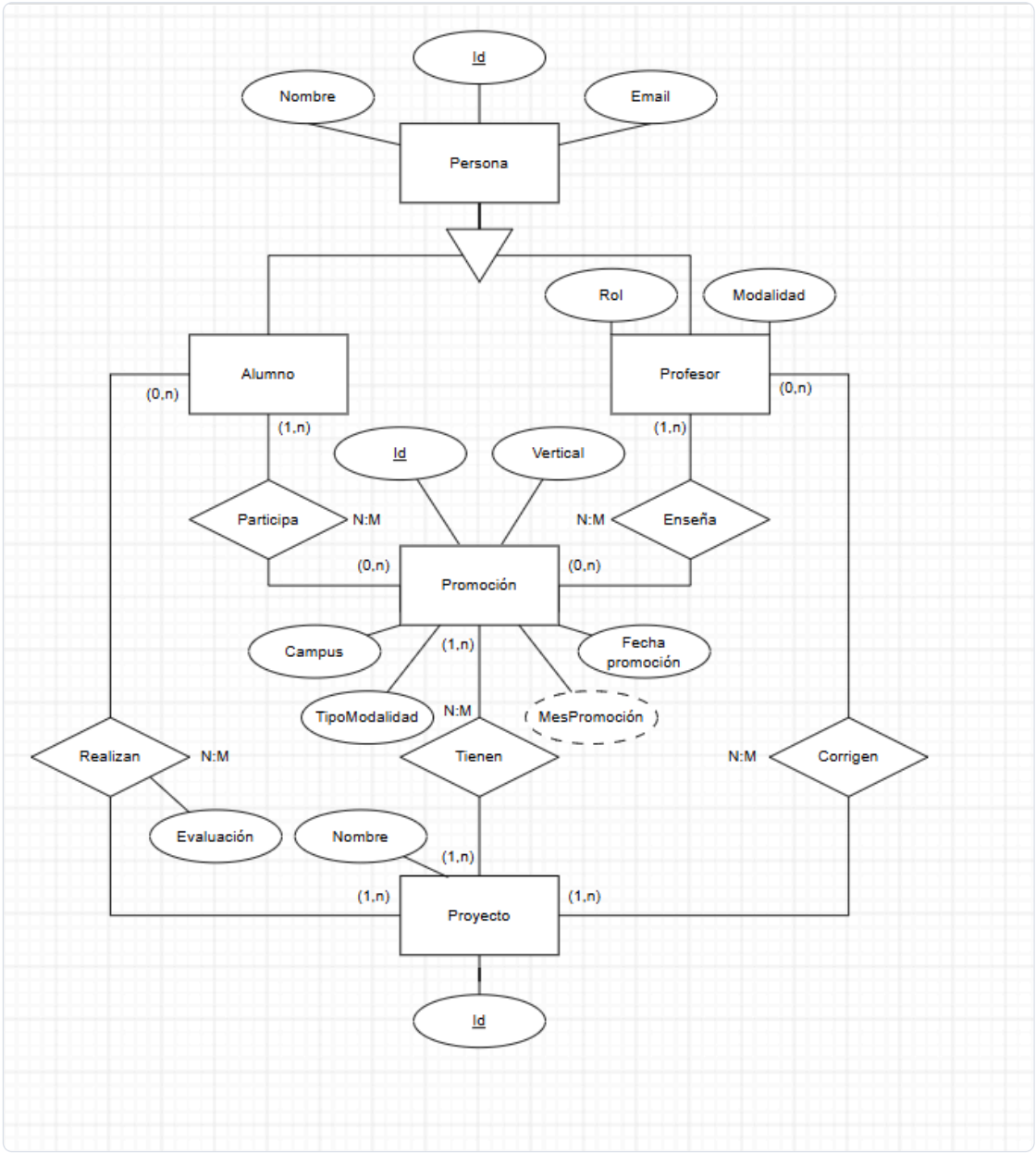


Figura 1 · Modelo Entidad-Relación del proyecto (draw.io)

Cómo se lee: rectángulo = entidad · elipse = atributo (subrayado = identificador, futura PK) · rombo = relación · triángulo = herencia (Persona → Alumno / Profesor, parcial y solapada) · elipse discontinua = atributo derivado (**MesPromoción** se calcula a partir de la fecha) · elipse colgando de un rombo = atributo de la relación (**Evaluación** no es del alumno ni del proyecto, sino de "este alumno en este proyecto") · (mín, máx) = cardinalidad de cada lado.

Algunas cardinalidades explicadas

- **Participa, lado Promoción (0,n):** una promoción puede existir sin alumnos. Pasa en los datos: las promociones 5 y 6 solo aparecen en el claustro.
- **Corrigen, lado Proyecto (1,n):** todo proyecto lo corrige al menos un profesor.
- **Modalidad en dos sitios:** la del profesor (en una misma promoción hay TA presenciales y LI online) y **TipoModalidad** en la promoción, que es la general.

3 · Paso 2: Modelo lógico (relacional)

El modelo lógico traduce el diagrama E/R a **tablas**, con sus columnas, claves primarias (PK) y claves foráneas (FK). Como vamos a usar PostgreSQL, que es una base de datos relacional, el modelo lógico es el **modelo relacional**.

3.1 Reglas para pasar del E/R a tablas

En el E/R hay…	En el modelo relacional se convierte en…
Una **entidad**	Una tabla. Su identificador pasa a ser la **PK**.
Un **atributo**	Una columna de esa tabla. Un atributo **derivado** no se guarda: se calcula al consultar.
Una **herencia**	Una tabla por subclase. Su PK es el id de la superclase, que a la vez es **FK** a ella (relación 1:1).
Una relación **1:N**	No crea tabla: la PK del lado "1" se copia como **FK** en la tabla del lado "N".
Una relación **N:M**	Crea una **tabla intermedia** con las PK de las dos entidades. Juntas forman su PK, y cada una es FK a su tabla. Los atributos de la relación (como Evaluación) van en esta tabla.

3.2 Aplicación a nuestro modelo

Elemento del E/R	Tabla resultante
Entidad Persona	`persona`
Subclase Alumno	`alumnos` (PK y FK `id_persona`)
Subclase Profesor	`profesor` (PK y FK `id_persona`, más rol y modalidad)
Entidad Promoción	`promocion` (sin MesPromoción, que es derivado)
Entidad Proyecto	`proyecto`
Relación N:M Participa	`alumnosPromocion`
Relación N:M Enseña	`profesorPromocion`
Relación N:M Realizan (+ Evaluación)	`alumnoProyecto` (con la columna `evaluacion`)
Relación N:M Tienen	`promocionProyecto`
Relación N:M Corrigen	`profesorProyecto`

En total: **3 entidades + 2 subclases + 5 tablas intermedias = 10 tablas**.

3.3 Esquema relacional (en texto)

Formato: tabla (columnas). PK clave primaria · FK clave foránea.

Tabla	Columnas
`persona`	**id** PK, nombre, email
`alumnos`	**id_persona** PK FK → persona
`profesor`	**id_persona** PK FK → persona, rol, modalidad
`promocion`	**id** PK, fechaPromocion, campus, vertical, tipoModalidad
`proyecto`	**id** PK, nombreProyecto
`alumnosPromocion`	**id_alumno** PK FK → alumnos, **id_promocion** PK FK → promocion
`profesorPromocion`	**id_profesor** PK FK → profesor, **id_promocion** PK FK → promocion
`alumnoProyecto`	**id_alumno** PK FK → alumnos, **id_proyecto** PK FK → proyecto, evaluacion
`promocionProyecto`	**id_promocion** PK FK → promocion, **id_proyecto** PK FK → proyecto
`profesorProyecto`	**id_profesor** PK FK → profesor, **id_proyecto** PK FK → proyecto

3.4 Diagrama del modelo relacional

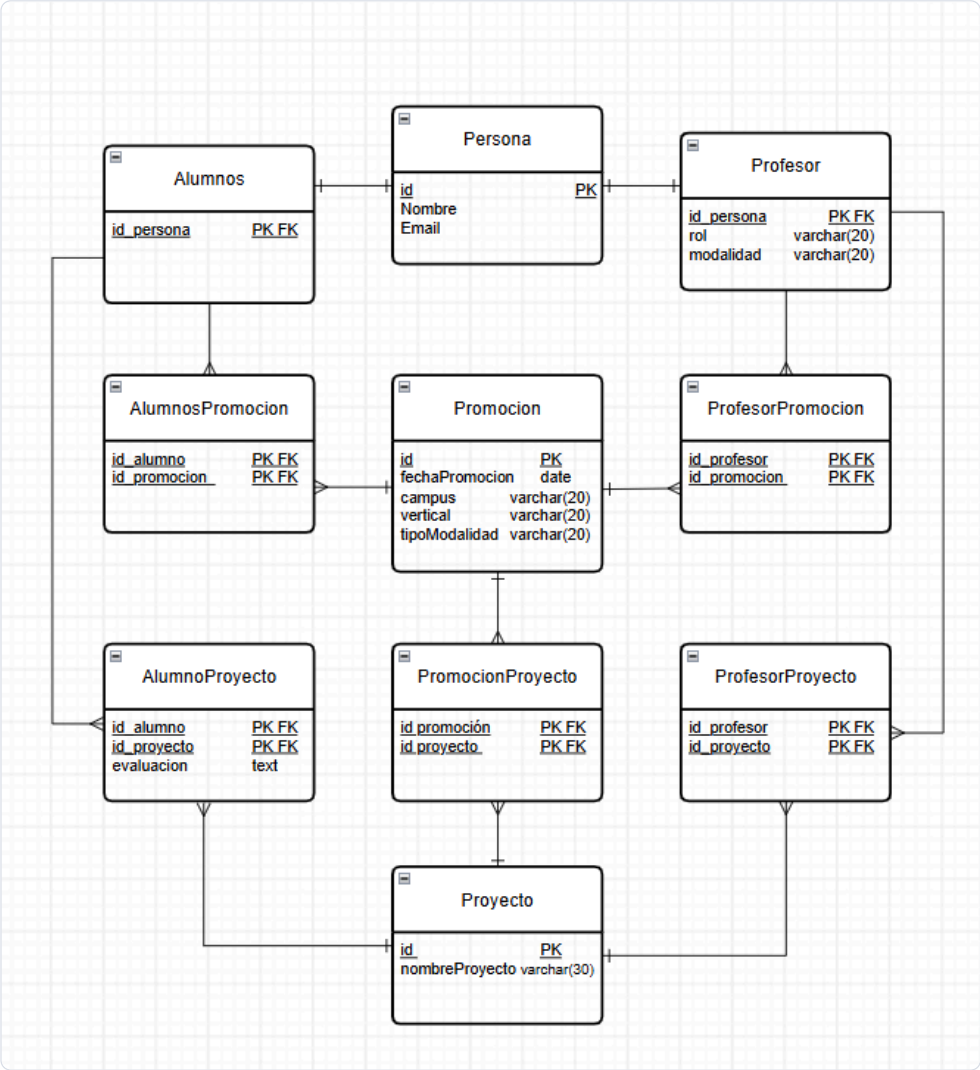


Figura 2 · Modelo lógico (relacional) del proyecto en draw.io: 10 tablas, con sus claves primarias (PK) y foráneas (FK)

> **Cómo leer la herencia y una tabla intermedia**
> En `alumnos`, la columna `id_persona` es a la vez PK y FK: el alumno 1 *es* la persona 1 (Jafet Casals). `alumnoProyecto` tiene 260 filas: una por cada pareja alumno–proyecto (52 alumnos × 5 proyectos). La fila `(1, 2, 'No Apto')` significa: "el alumno 1 (Jafet Casals), en el proyecto 2 (EDA), tiene No Apto". La PK compuesta `(id_alumno, id_proyecto)` impide que un alumno tenga dos notas en el mismo proyecto.

Tipos de datos elegidos

Columna	Tipo	Motivo
`id` de persona, promoción y proyecto	INTEGER autoincremental	La base de datos genera el número sola al insertar.
`id_persona` en alumnos y profesor	INTEGER (no autoincremental)	Copia el id de la persona; no se inventa uno nuevo.
`fechaPromocion`	DATE	Permite ordenar, filtrar y sacar el mes (no es un texto).
`nombre`, `campus`, `vertical`, `rol`, `nombreProyecto`	VARCHAR(n)	Texto corto con un tamaño máximo.
`email`, `evaluacion`	TEXT	Texto sin límite de longitud.
`modalidad`, `tipoModalidad`	ENUM en MySQL · VARCHAR + CHECK en PostgreSQL	Solo admite valores de una lista (presencial, online; y en la promoción, también híbrida).

4 · Paso 3: Verificación de la normalización

El enunciado pide analizar los datos y normalizarlos. Ya lo hicimos en el Paso 1 para diseñar el modelo; ahora **comprobamos sobre las tablas finales** que cumplen las tres formas normales. Si alguna fallara, tendríamos que volver a tocar el E/R.

Tabla	1FN valores únicos, sin grupos repetidos	2FN todo depende de la PK entera	3FN nada depende de algo que no sea la PK
`persona`	✓	✓ PK simple	✓ nombre y email se guardan una sola vez
`alumnos`	✓	✓ solo tiene la PK	✓
`profesor`	✓	✓ PK simple	✓ rol y modalidad son del profesor
`promocion`	✓	✓ PK simple	✓ sin el mes, que es derivado
`proyecto`	✓	✓ PK simple	✓
`alumnoProyecto`	✓ una nota por fila	✓ la evaluación depende de alumno *y* proyecto	✓
`alumnosPromocion`, `profesorPromocion`, `promocionProyecto`, `profesorProyecto`	✓	✓ solo tienen la PK	✓

> **Conclusión**
> Las 10 tablas cumplen la 3FN, así que el diseño de los pasos 1 y 2 es correcto.

Cosas que revisamos durante la verificación

> **Error que encontramos y corregimos: la promoción guardada dos veces**
> En una versión anterior, la tabla de alumnos tenía una columna `id_promocion` (relación 1:N) **y además** existía la tabla `alumnosPromocion`. Así, la promoción de cada alumno estaba guardada en dos sitios, que es justo la redundancia que queremos evitar. Lo corregimos dejando solo la relación N:M.

> **Mejora del diseño: de alumnos y claustro a Persona**
> En la primera versión, alumnos y claustro eran dos tablas independientes con nombre y email cada una. Si un alumno pasaba a ser TA, había que darlo de alta otra vez en el claustro, con su nombre repetido: una anomalía de actualización. Con `persona` y la herencia solapada, la persona existe una sola vez y puede ser alumno, profesor o las dos cosas.

> **Atributo derivado: MesPromoción**
> El mes se puede sacar de `fechaPromocion`, así que no lo guardamos: si estuviera en una columna y alguien cambiara la fecha, podrían quedar contradictorios. Se calcula al consultar con `EXTRACT(MONTH FROM fechaPromocion)` (en MySQL, `MONTH(fecha)`). Por la misma razón quitamos el año que tenía la primera versión.

Integridad de los datos

- **Claves primarias** en todas las tablas: no puede haber dos filas iguales.
- **Claves foráneas** en las subclases y en las tablas intermedias (12 en total): no se puede poner una nota a un alumno que no existe, ni dar de alta un profesor que no sea una persona.
- **PK compuestas** en las tablas N:M: un alumno no puede estar dos veces en la misma promoción, ni tener dos notas en el mismo proyecto.
- **Tipos adecuados**: fechas como `DATE` y modalidades restringidas a valores válidos.

5 · Paso 4: Creación de la base de datos

Primero la creamos en **MySQL Workbench** (en local) para probar el diseño. Después la pasamos a **PostgreSQL**, que es lo que pide el enunciado, y la alojamos en **Supabase**, un servicio gratuito de PostgreSQL en la nube (el enunciado sugiere Render; los dos sirven porque los dos son PostgreSQL).

5.1 Script de creación de tablas

El orden importa, porque una FK solo puede apuntar a una tabla que ya existe: primero las tablas **sin FK** (persona, promoción, proyecto), después las **subclases** (alumnos y profesor, que apuntan a persona) y al final las **intermedias**. Extracto del script MySQL:

```
CREATE TABLE persona (
    id INTEGER PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(50),
    email TEXT
);
-- Subclases: misma clave que persona (PK y FK a la vez). Herencia parcial y solapada.
CREATE TABLE alumnos (
    id_persona INTEGER PRIMARY KEY,
    FOREIGN KEY (id_persona) REFERENCES persona(id)
);
CREATE TABLE profesor (
    id_persona INTEGER PRIMARY KEY,
    rol VARCHAR(20),
    modalidad ENUM('presencial', 'online'),
    FOREIGN KEY (id_persona) REFERENCES persona(id)
);
-- MesPromocion es derivado: no se guarda, se calcula con MONTH(fechaPromocion).
CREATE TABLE promocion (
    id INTEGER PRIMARY KEY AUTO_INCREMENT,
    fechaPromocion DATE,
    campus VARCHAR(20),
    vertical VARCHAR(20),
    tipoModalidad ENUM('presencial', 'online', 'hibrida')
);
-- Tabla intermedia N:M con atributo propio (evaluacion)
CREATE TABLE alumnoProyecto (
    id_alumno INTEGER,
    id_proyecto INTEGER,
    evaluacion TEXT,
    PRIMARY KEY (id_alumno, id_proyecto),
    FOREIGN KEY (id_alumno) REFERENCES alumnos(id_persona),
    FOREIGN KEY (id_proyecto) REFERENCES proyecto(id)
);
-- ... y así proyecto y el resto de intermedias (alumnosPromocion, profesorPromocion, promocionProyecto, profesorProyecto)
```

5.2 Carga de los datos (INSERT)

Los `INSERT` se generan a partir de los 5 CSV, aplicando la normalización del Paso 1:

1. **Promociones:** cada combinación distinta de fecha + campus + vertical es una promoción (4 de los CSV de alumnos + 2 que solo aparecen en el claustro = **6**).
2. **Proyectos:** se sacan de las cabeceras `Proyecto_X` (**10**).
3. **Personas:** los 52 alumnos (ids 1–52) y los 10 profesores (ids 53–62) = **62**. Los profesores no traen email en `claustro.csv`, así que queda vacío.
4. **Subclases:** `alumnos` (**52**) y `profesor` con su rol y modalidad (**10**).
5. **Participa y Enseña:** la promoción de cada alumno en `alumnosPromocion` (**52**) y la de cada profesor en `profesorPromocion` (**10**).
6. **Evaluaciones:** cada celda Apto / No Apto pasa a ser una fila de `alumnoProyecto` (52 × 5 = **260**).
7. **Proyectos de cada promoción:** `promocionProyecto` (4 promociones × 5 = **20**).
8. **Correcciones:** `profesorProyecto` (**18**). Los CSV no dicen quién corrige cada proyecto, así que estas filas son datos de ejemplo (ver sección 8).

```
INSERT INTO persona (id, nombre, email) VALUES
    (1, 'Jafet Casals', 'Jafet_Casals@gmail.com'),
    ...
    (62, 'María Dolores Diaz', NULL);
INSERT INTO profesor (id_persona, rol, modalidad) VALUES
    (53, 'TA', 'presencial'),   -- Noa Yáñez
    ...
INSERT INTO alumnoProyecto (id_alumno, id_proyecto, evaluacion) VALUES
    (1, 1, 'Apto'),      -- Jafet Casals · HLF
    (1, 2, 'No Apto'),   -- Jafet Casals · EDA
    ...
```

5.3 De MySQL a PostgreSQL

MySQL y PostgreSQL usan casi el mismo SQL, pero hay diferencias que hacen fallar el script si no se adaptan:

En MySQL	En PostgreSQL	Por qué
`DROP/CREATE DATABASE` , `USE` , `SET NAMES`	`CREATE SCHEMA v2` y `SET search_path TO v2`	Supabase ya nos da una base de datos. Usamos un esquema propio para no chocar con las tablas de la primera versión.
`AUTO_INCREMENT`	`GENERATED BY DEFAULT AS IDENTITY`	Es la forma estándar de PostgreSQL de numerar automáticamente.
`ENUM('presencial','online')`	`VARCHAR(20) CHECK (modalidad IN ('presencial','online'))`	PostgreSQL no admite ENUM dentro del CREATE TABLE.
`MONTH(fecha)`	`EXTRACT(MONTH FROM fecha)`	Para calcular MesPromoción.
—	`DROP TABLE IF EXISTS … CASCADE` al principio	Para poder ejecutar el script varias veces sin errores.
—	`BEGIN; … COMMIT;`	Si algo falla, no se queda nada a medias.
—	`SELECT setval(...)` al final	Como insertamos los id a mano, hay que avisar al contador de que siga por el 63 (y no por el 1).

Un detalle: PostgreSQL guarda los nombres sin comillas en minúsculas, así que `alumnoProyecto` aparece como `alumnoproyecto` . Las consultas funcionan igual escribiéndolo de cualquiera de las dos formas.

PostgreSQL ProyectoGrupal_v2_PostgreSQL.sql · Supabase

```
7  BEGIN;
8
9  CREATE SCHEMA IF NOT EXISTS v2;
10 SET search_path TO v2;
11
12 DROP TABLE IF EXISTS profesorProyecto, promocionProyecto, alumnoProyecto,
13     profesorPromocion, alumnosPromocion, proyecto, promocion, profesor,
14     alumnos, persona CASCADE;
15
16 -- ---------- Entidades ----------
17 CREATE TABLE persona (
18     id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
19     nombre VARCHAR(50),
20     email TEXT
21 );
22
23 -- Subclases: misma clave que persona (PK y FK a la vez). Herencia parcial y solapada.
24 CREATE TABLE alumnos (
25     id_persona INTEGER PRIMARY KEY,
26     FOREIGN KEY (id_persona) REFERENCES persona(id)
27 );
28
29 CREATE TABLE profesor (
30     id_persona INTEGER PRIMARY KEY,
31     rol VARCHAR(20),
32     modalidad VARCHAR(20) CHECK (modalidad IN ('presencial', 'online')),
33     FOREIGN KEY (id_persona) REFERENCES persona(id)
34 );
35
36 -- MesPromocion es derivado: no se guarda, se calcula con EXTRACT(MONTH FROM fechaPromocion).
37 CREATE TABLE promocion (
38     id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
39     fechaPromocion DATE,
40     campus VARCHAR(20),
41     vertical VARCHAR(20),
42     tipoModalidad VARCHAR(20) CHECK (tipoModalidad IN ('presencial', 'online', 'hibrida'))
43 );
44
45 CREATE TABLE proyecto (
46     id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
47     nombreProyecto VARCHAR(30)
48 );
```

Figura 3 · Inicio del script de PostgreSQL. En amarillo, lo que cambia respecto a MySQL

5.4 Paso a paso en Supabase

1. 1 Entrar en **supabase.com**, crear un proyecto nuevo (nombre, contraseña de la base de datos y región).
2. 2 En el menú lateral, abrir **SQL Editor** → **New query**.
3. 3 Pegar el contenido completo de `ProyectoGrupal_v2_PostgreSQL.sql` y pulsar **Run**.
4. 4 Comprobar el resultado de la consulta de recuento (abajo). Las tablas aparecen en **Table Editor**, eligiendo el esquema `v2` .
5. 5 Para conectarse desde otras aplicaciones (Python, DBeaver, Power BI…), pulsar **Connect** y copiar la cadena de conexión. La contraseña no se comparte en el repositorio.

Resultado en Supabase: el *Schema Visualizer* (menú Database, esquema `v2`) dibuja automáticamente las 10 tablas y sus relaciones a partir de las claves foráneas. Es el mismo modelo relacional de la Figura 2.

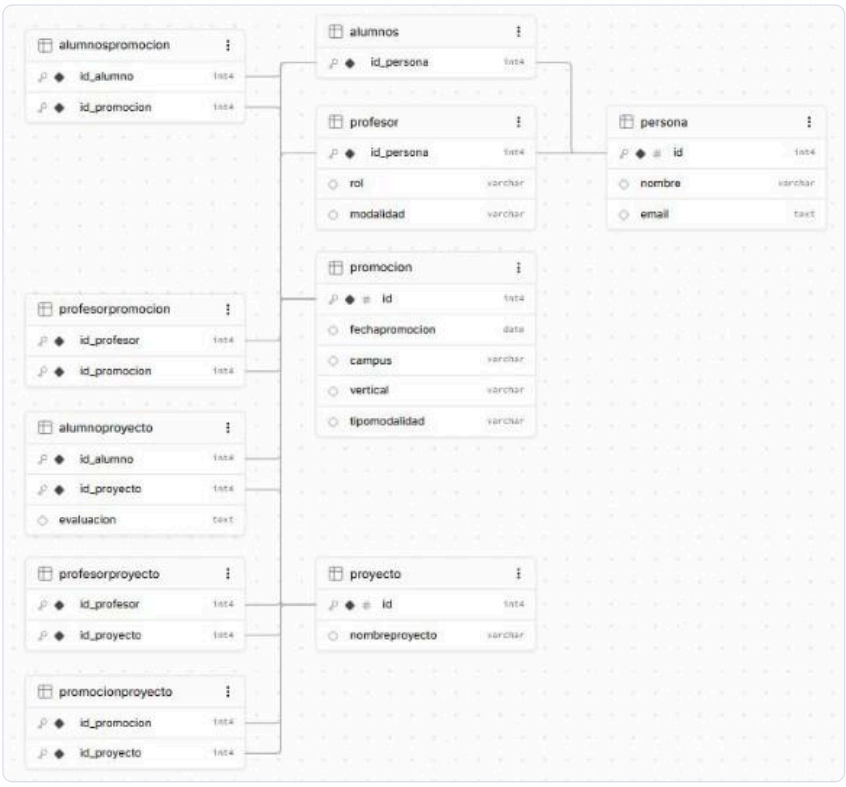


Figura 4 · Schema Visualizer de Supabase con las 10 tablas del esquema v2

El mismo esquema, obtenido con la **ingeniería inversa de MySQL Workbench** (*Database → Reverse Engineer*), se lee mejor: muestra el tipo de cada columna, marca las claves primarias y foráneas con iconos, y dibuja las relaciones con la notación "pata de gallo" (una raya = 1, tres patas = N). Entre persona y sus subclases, la relación es 1 a 1.

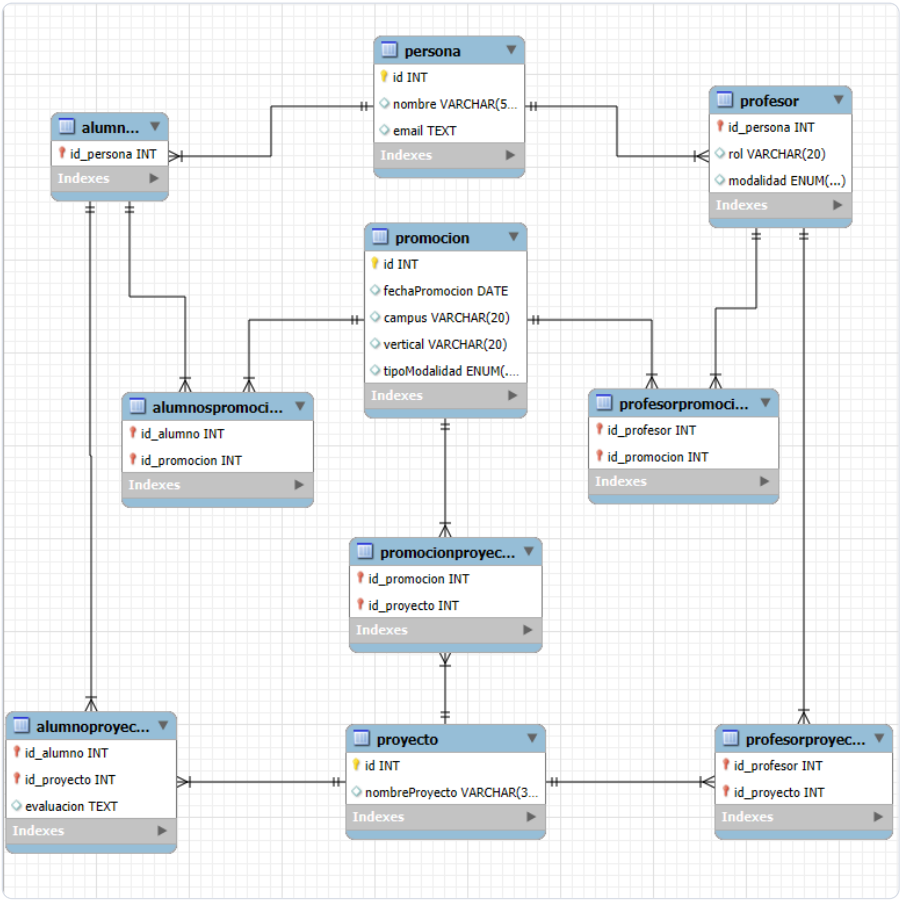


Figura 5 · Ingeniería inversa del esquema en MySQL Workbench

5.5 Comprobación: ¿se han cargado todos los datos?

```sql
-- Recuento de filas del modelo v2
SELECT 'persona' AS tabla, COUNT(*) AS filas FROM v2.persona
UNION ALL SELECT 'alumnos', COUNT(*) FROM v2.alumnos
UNION ALL SELECT 'profesor', COUNT(*) FROM v2.profesor
UNION ALL SELECT 'promocion', COUNT(*) FROM v2.promocion
UNION ALL SELECT 'proyecto', COUNT(*) FROM v2.proyecto
UNION ALL SELECT 'alumnosPromocion', COUNT(*) FROM v2.alumnosPromocion
UNION ALL SELECT 'profesorPromocion', COUNT(*) FROM v2.profesorPromocion
UNION ALL SELECT 'alumnoProyecto', COUNT(*) FROM v2.alumnoProyecto
UNION ALL SELECT 'promocionProyecto', COUNT(*) FROM v2.promocionProyecto
UNION ALL SELECT 'profesorProyecto', COUNT(*) FROM v2.profesorProyecto;
```

Results Chart

tabla	filas
persona	62
alumnos	52
profesor	10
promocion	6
proyecto	10
alumnosPromocion	52
profesorPromocion	10
alumnoProyecto	260
promocionProyecto	20
profesorProyecto	18

Figura 6 · Consulta de recuento ejecutada en el SQL Editor de Supabase

Total: 500 filas en 10 tablas

Los CSV tienen 62 filas (52 alumnos y 10 profesores). Salen 500 porque, al normalizar, cada dato va a su tabla y las relaciones se guardan aparte: por ejemplo, las 5 notas de cada alumno pasan a ser 5 filas (52 × 5 = 260).

6 · Consultas de demostración

Estas consultas son las que enseñaremos en la presentación para demostrar que la base de datos funciona. Todas usan `JOIN` para volver a juntar la información que la normalización repartió en varias tablas. Están en `sql/consultas_demo.sql`, que empieza con `SET search_path TO v2;`.

Consulta 1 · ¿Cuántos alumnos tiene cada promoción?

```sql
SELECT pr.id, pr.vertical, pr.campus, pr.fechaPromocion,
       EXTRACT(MONTH FROM pr.fechaPromocion) AS mesPromocion,
       COUNT(apr.id_alumno) AS num_alumnos
FROM promocion pr
    LEFT JOIN alumnosPromocion apr ON pr.id = apr.id_promocion
GROUP BY pr.id, pr.vertical, pr.campus, pr.fechaPromocion
ORDER BY pr.id;
```

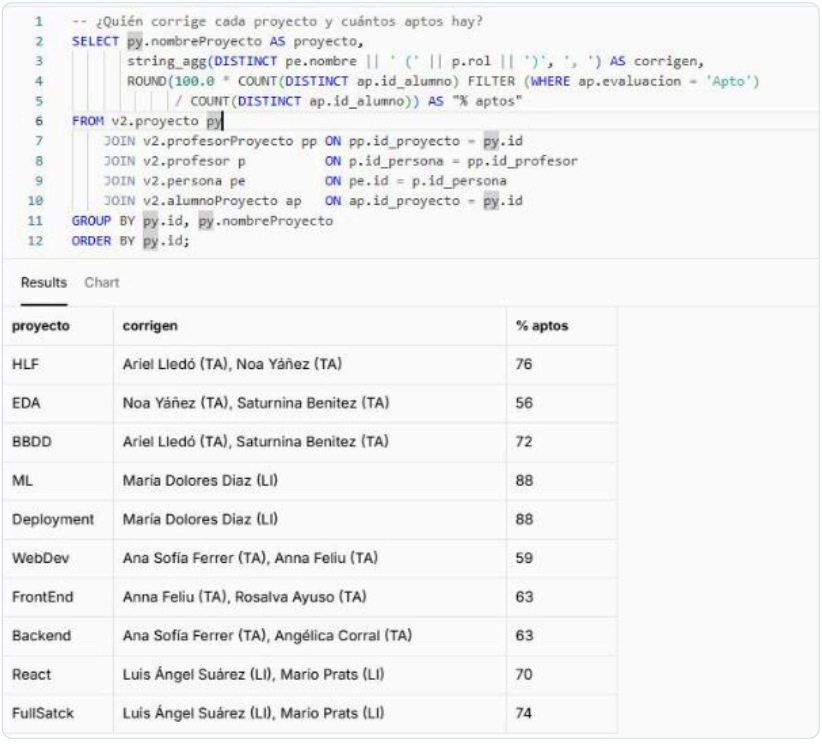
id	vertical	campus	fechaPromocion	mesPromocion	num_alumnos
1	DS	Madrid	2023-09-18	9	15
2	DS	Madrid	2024-02-12	2	10
3	FS	Madrid	2023-09-18	9	14
4	FS	Valencia	2024-02-12	2	13
5	FS	Valencia	2023-09-18	9	0
6	FS	Madrid	2024-02-12	2	0

6 filas

Usamos `LEFT JOIN` para que también salgan las promociones 5 y 6, que tienen profesores pero aún no tienen alumnos. Esto es justo lo que la tabla original no permitía (anomalía de inserción). El mes no está guardado: lo calcula `EXTRACT`.

Consulta 2 · Quién corrige cada proyecto y porcentaje de aptos

```sql
SELECT py.nombreProyecto AS proyecto,
       string_agg(DISTINCT pe.nombre || ' (' || p.rol || ')', ', ') AS corrigen,
       ROUND(100.0 * COUNT(DISTINCT ap.id_alumno) FILTER (WHERE ap.evaluacion = 'Apto')
             / COUNT(DISTINCT ap.id_alumno)) AS porcentaje_aptos
FROM proyecto py
    JOIN profesorProyecto pp ON pp.id_proyecto = py.id
    JOIN profesor p          ON p.id_persona = pp.id_profesor
    JOIN persona pe          ON pe.id = p.id_persona
    JOIN alumnoProyecto ap   ON ap.id_proyecto = py.id
GROUP BY py.id, py.nombreProyecto
ORDER BY py.id;
```



```sql
-- ¿Quién corrige cada proyecto y cuántos aptos hay?
SELECT py.nombreProyecto AS proyecto,
       string_agg(DISTINCT pe.nombre || ' (' || p.rol || ')', ', ') AS corrigen,
       ROUND(100.0 * COUNT(DISTINCT ap.id_alumno) FILTER (WHERE ap.evaluacion = 'Apto')
             / COUNT(DISTINCT ap.id_alumno)) AS "% aptos"
FROM v2.proyecto py
    JOIN v2.profesorProyecto pp ON pp.id_proyecto = py.id
    JOIN v2.profesor p          ON p.id_persona = pp.id_profesor
    JOIN v2.persona pe          ON pe.id = p.id_persona
    JOIN v2.alumnoProyecto ap   ON ap.id_proyecto = py.id
GROUP BY py.id, py.nombreProyecto
ORDER BY py.id;
```

Results Chart

proyecto	corrigen	% aptos
HLF	Ariel Lledó (TA), Noa Yáñez (TA)	76
EDA	Noa Yáñez (TA), Saturnina Benitez (TA)	56
BBDD	Ariel Lledó (TA), Saturnina Benitez (TA)	72
ML	María Dolores Diaz (LI)	88
Deployment	María Dolores Diaz (LI)	88
WebDev	Ana Sofía Ferrer (TA), Anna Feliu (TA)	59
FrontEnd	Anna Feliu (TA), Rosalva Ayuso (TA)	63
Backend	Ana Sofía Ferrer (TA), Angélica Corral (TA)	63
React	Luis Ángel Suárez (LI), Mario Prats (LI)	70
FullSatck	Luis Ángel Suárez (LI), Mario Prats (LI)	74

Figura 7 · Consulta 2 ejecutada en Supabase

EDA es el proyecto con más suspensos (56 % de aptos); ML y Deployment, los que más aprueban (88 %). La columna **corrigen** solo se puede sacar con el modelo nuevo; sus datos son de ejemplo, porque los CSV no los traen.

Consulta 3 · Ficha de un alumno: promoción y notas

```sql
SELECT pe.nombre, pr.vertical, pr.campus, py.nombreProyecto AS proyecto, ap.evaluacion
FROM persona pe
    JOIN alumnos a            ON a.id_persona = pe.id
    JOIN alumnosPromocion apr ON apr.id_alumno = a.id_persona
    JOIN promocion pr         ON pr.id = apr.id_promocion
    JOIN alumnoProyecto ap    ON ap.id_alumno = a.id_persona
    JOIN proyecto py          ON py.id = ap.id_proyecto
WHERE pe.nombre LIKE 'Jafet%'
ORDER BY py.id;
```

nombre	vertical	campus	proyecto	evaluacion
Jafet Casals	DS	Madrid	HLF	Apto
Jafet Casals	DS	Madrid	EDA	No Apto
Jafet Casals	DS	Madrid	BBDD	Apto
Jafet Casals	DS	Madrid	ML	Apto
Jafet Casals	DS	Madrid	Deployment	Apto

5 filas

Con 5 `JOIN` reconstruimos la fila original del CSV, pero ahora a partir de datos sin redundancia. El nombre sale de `persona`.

Consulta 4 · Profesorado de cada promoción y su modalidad

```sql
SELECT pr.id AS promocion, pr.vertical, pr.campus, pr.fechaPromocion,
       pe.nombre AS profesor, p.rol, p.modalidad
FROM promocion pr
    JOIN profesorPromocion pp ON pp.id_promocion = pr.id
    JOIN profesor p           ON p.id_persona = pp.id_profesor
    JOIN persona pe           ON pe.id = p.id_persona
ORDER BY pr.id, p.rol DESC, pe.nombre;
```

promocion	vertical	campus	fechaPromocion	profesor	rol	modalidad
1	DS	Madrid	2023-09-18	Ariel Lledó	TA	presencial
1	DS	Madrid	2023-09-18	Noa Yáñez	TA	presencial
1	DS	Madrid	2023-09-18	Saturnina Benitez	TA	presencial
1	DS	Madrid	2023-09-18	María Dolores Diaz	LI	online
3	FS	Madrid	2023-09-18	Anna Feliu	TA	presencial
3	FS	Madrid	2023-09-18	Luis Ángel Suárez	LI	online
4	FS	Valencia	2024-02-12	Ana Sofía Ferrer	TA	presencial
4	FS	Valencia	2024-02-12	Mario Prats	LI	online
5	FS	Valencia	2023-09-18	Rosalva Ayuso	TA	presencial
6	FS	Madrid	2024-02-12	Angélica Corral	TA	presencial

10 filas

Consulta 5 · Profesores que corrigen proyectos de una promoción en la que no enseñan

```sql
SELECT DISTINCT pe.nombre AS profesor, p.rol, py.nombreProyecto AS proyecto
FROM profesorProyecto pp
    JOIN profesor p  ON p.id_persona = pp.id_profesor
    JOIN persona pe  ON pe.id = p.id_persona
    JOIN proyecto py ON py.id = pp.id_proyecto
WHERE NOT EXISTS (
    SELECT 1
    FROM profesorPromocion ppr
        JOIN promocionProyecto prp ON prp.id_promocion = ppr.id_promocion
    WHERE ppr.id_profesor = pp.id_profesor AND prp.id_proyecto = pp.id_proyecto
)
ORDER BY pe.nombre;
```

profesor	rol	proyecto
Angélica Corral	TA	Backend
Rosalva Ayuso	TA	FrontEnd

2 filas

Rosalva (promoción 5) y Angélica (promoción 6) no tienen alumnos en su promoción y ayudan a corregir proyectos de otra. Con el primer modelo esto no se podía guardar: solo se podía suponer que corregía el profesorado de la misma promoción.

Consulta 6 · Personas que son alumno y profesor a la vez

```sql
-- Con los datos de los CSV sale vacía. Probamos antes a hacer TA a Jafet:
INSERT INTO profesor (id_persona, rol, modalidad) VALUES (1, 'TA', 'presencial');

SELECT pe.id, pe.nombre
FROM persona pe
    JOIN alumnos a  ON a.id_persona = pe.id
    JOIN profesor p ON p.id_persona = pe.id;
```

id	nombre
1	Jafet Casals

1 fila

Es la herencia **solapada** en acción: Jafet está en `alumnos` y en `profesor`, pero su nombre y su email siguen guardados una sola vez, en `persona`.

Consulta 7 · Resumen por campus y vertical

```sql
SELECT pr.campus, pr.vertical,
       COUNT(DISTINCT apr.id_alumno)  AS alumnos,
       COUNT(DISTINCT pp.id_profesor) AS profesores
FROM promocion pr
    LEFT JOIN alumnosPromocion apr ON pr.id = apr.id_promocion
    LEFT JOIN profesorPromocion pp ON pr.id = pp.id_promocion
GROUP BY pr.campus, pr.vertical
ORDER BY pr.campus, pr.vertical;
```

campus	vertical	alumnos	profesores
Madrid	DS	25	4
Madrid	FS	14	3
Valencia	FS	13	3

3 filas

`COUNT(DISTINCT …)` evita contar dos veces a la misma persona al unir dos tablas a la vez.

7 · Escalabilidad: cómo crece la base de datos

El enunciado pide que la base de datos pueda crecer en campus, verticales, promociones, modalidades, etc. Gracias a la normalización, casi todo se hace **añadiendo filas, sin cambiar la estructura**:

Abrir un campus nuevo (Barcelona) con una promoción de Data Science

```
INSERT INTO promocion (fechaPromocion, campus, vertical, tipoModalidad)
VALUES ('2025-09-15', 'Barcelona', 'DS', 'presencial');   -- el id se genera solo
```

Crear una vertical nueva (Ciberseguridad) con sus proyectos

```
INSERT INTO proyecto (nombreProyecto) VALUES ('Redes'), ('Pentesting');
INSERT INTO promocion (fechaPromocion, campus, vertical, tipoModalidad)
VALUES ('2025-10-01', 'Madrid', 'CS', 'online');
-- y se enlazan en promocionProyecto
```

Un alumno que hace un segundo bootcamp

```
-- Jafet (id 1) se apunta también a la promoción Full Stack de Madrid (id 3)
INSERT INTO alumnosPromocion (id_alumno, id_promocion) VALUES (1, 3);
```

Un alumno que pasa a ser profesor (TA)

```
-- Jafet ya es la persona 1: solo hay que añadirlo a profesor
INSERT INTO profesor (id_persona, rol, modalidad) VALUES (1, 'TA', 'presencial');
```

Esto es posible por la herencia **solapada**: no hace falta repetir su nombre ni su email.

Quiero añadir…	¿Cambia la estructura?	Cómo
Un campus o una promoción	No	Nueva fila en `promocion`
Un proyecto	No	Nueva fila en `proyecto` + `promocionProyecto`
Una vertical	No	Nuevo valor en `promocion.vertical`
Un profesor	No	Nueva fila en `persona` + `profesor` + `profesorPromocion`
Un alumno en dos bootcamps	No	Otra fila en `alumnosPromocion` (N:M)
Un alumno que también es TA	No	Una fila en `profesor` con su mismo id de persona
Quién corrige un proyecto	No	Nueva fila en `profesorProyecto`
Una modalidad nueva (p. ej. "telemática")	Sí	Hay que modificar el ENUM / CHECK (ver mejoras)
Aulas	Sí	Nueva tabla `aula` relacionada con promoción (ver mejoras)

8 · Decisiones, limitaciones y mejoras futuras

Decisiones tomadas

- **Normalizar antes del E/R**, porque partimos de datos existentes (ver sección 0).
- **Generalizar alumnos y profesores en Persona**, con herencia parcial y solapada, para no repetir nombre y email y permitir que una persona tenga los dos papeles.
- **Todas las relaciones N:M**, para que un alumno pueda estar en varias promociones y un profesor corregir proyectos de otras.
- **Guardar quién corrige (Corrigen)** en vez de suponerlo a partir de la promoción.
- **MesPromoción como atributo derivado**, calculado a partir de la fecha.
- **Crear las promociones 5 y 6** aunque no tengan alumnos, porque aparecen en el claustro.
- **Mantener los nombres tal como vienen en los CSV** (incluido `FullSatck`) para no alterar los datos de origen.
- **Supabase en lugar de Render**: los dos ofrecen PostgreSQL gratuito; el script funciona en ambos.

Datos que no venían en los CSV

> - **Quién corrige cada proyecto:** las 18 filas de `profesorProyecto` son **datos de ejemplo**. Criterio: los TA corrigen los primeros proyectos de su vertical y los LI los más avanzados; Rosalva y Angélica, que no tienen alumnos en su promoción, ayudan con proyectos de otra.
> - **El email de los profesores** no viene en `claustro.csv`, así que queda vacío (`NULL`).
> - **La modalidad de la promoción** no viene en ningún CSV; se ha supuesto `presencial` porque los alumnos tienen campus.

Mejoras que se podrían hacer

Mejora	Qué aporta
Tabla `modalidad` (id, tipo)	Añadir "Telemática" sería un simple INSERT, sin tocar la estructura, y la usarían profesor y promoción.
Tablas `campus` y `vertical`	Evitan errores de escritura ("Madrid" vs "madrid") y permiten guardar más datos de cada campus (dirección, ciudad...).
Tabla `aula`	El enunciado menciona las aulas como posible crecimiento.
Corrector de cada entrega	Corrigen une profesor con proyecto, no con la entrega de cada alumno. Con un `id_profesor` en `alumnoProyecto` sabríamos quién corrigió el EDA de Jafet.
`UNIQUE` en `persona.email` y `NOT NULL` en columnas obligatorias	Más integridad: no puede haber dos personas con el mismo email ni personas sin nombre.
`CHECK (evaluacion IN ('Apto','No Apto'))`	Impide valores de evaluación inválidos.
Activar RLS en Supabase	Hoy está desactivado: con la clave pública del proyecto se podrían cambiar datos.
Corregir `FullSatck` → `FullStack`	Limpieza de datos de origen.

Conclusión

Partimos de 5 CSV con datos repetidos y columnas que significaban lo mismo, y llegamos a una base de datos de **10 tablas en 3FN**, con herencia entre persona, alumno y profesor, claves primarias y foráneas que garantizan la integridad, **500 filas** cargadas, alojada en PostgreSQL (Supabase) y preparada para crecer en campus, promociones, proyectos y profesorado sin cambiar su estructura.

A · Anexo: ficheros entregados

Carpeta / fichero del repositorio	Contenido
`README.md`	Resumen del proyecto, equipo y cómo reproducirlo
`enunciado/Proyecto_BBDD.md`	Enunciado del proyecto
`data/`	Datos de partida: `clase_1.csv` … `clase_4.csv` y `claustro.csv`
`diagramas/`	Modelo E/R (Figura 1), modelo lógico (Figura 2), ingeniería inversa en Workbench y esquema en Supabase
`sql/mysql/ProyectoGrupal_v2_MySQL.sql`	Creación de tablas y carga de datos para MySQL Workbench
`sql/postgresql/ProyectoGrupal_v2_PostgreSQL.sql`	Script completo para Supabase / Render: esquema v2, tablas, datos y comprobación
`sql/consultas_demo.sql`	Las consultas de demostración de la sección 6
`python/carga_alumnos_pandas.py`	Carga alternativa de los CSV con pandas + SQLAlchemy (prueba en SQLite, hecha con la primera versión del modelo)
`docs/`	Esta memoria en PDF, la presentación y las capturas
`sql/v1/` , `diagramas/v1/` , `docs/capturas/v1/`	La primera versión del modelo (8 tablas, sin herencia), que se conserva como referencia